myTutor Developer Guide

Architecture, prerequisites, setup, configuration and operations

Version 1.0 · 28 October 2026
Author: Naw Aung (81017125), UC Executive MBA · MBAM601 Digital Experiment
Repository: github.com/henrynawaung/mytutor

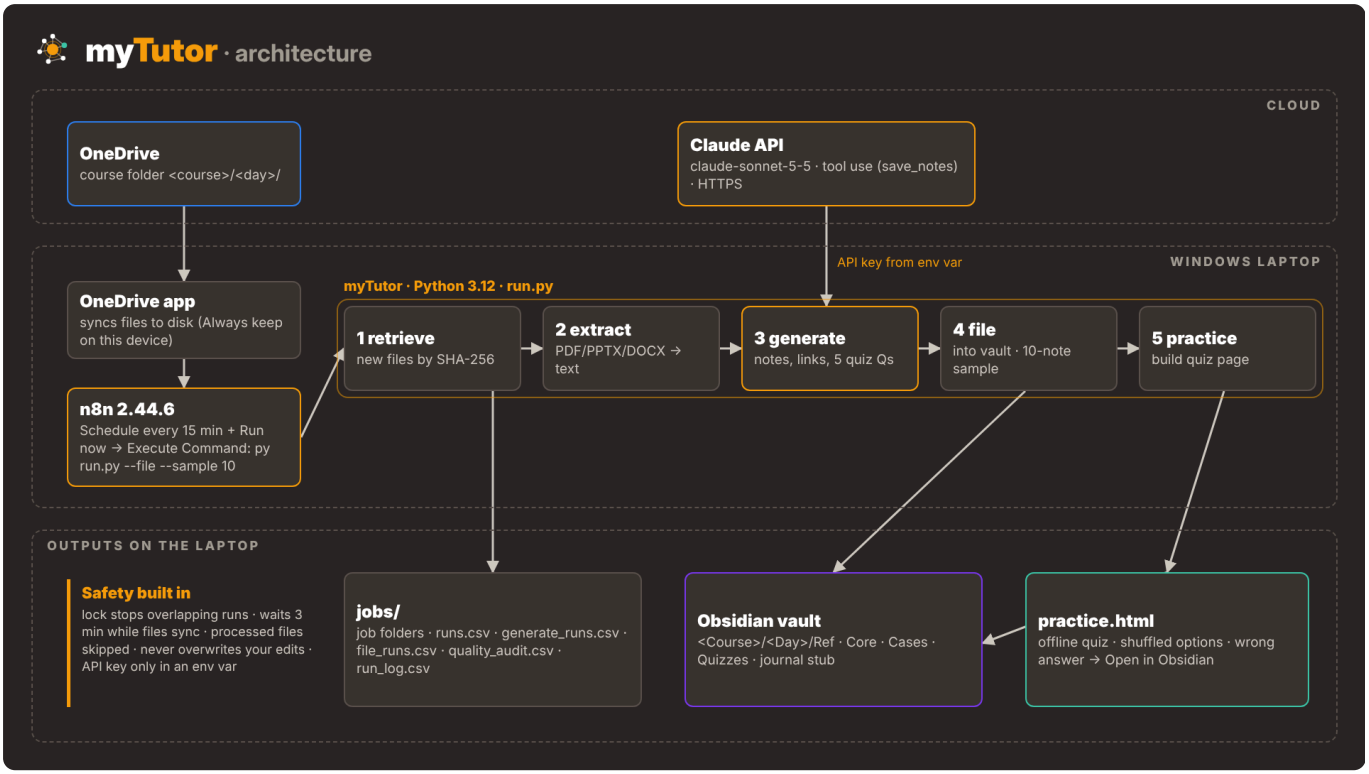
1. Overview
2. Architecture
3. Prerequisites
4. Technology stack and versions
5. Installation and setup
6. Configuration reference
7. Running and commands
8. Data, logs and folders
9. Testing
10. Security and responsible use
11. Troubleshooting
12. Known limits and roadmap
13. Key prompts and the manual test template

1. Overview

myTutor turns course files saved in OneDrive (lecture slides, readings, cases) into linked Obsidian notes and an offline practice quiz. n8n checks the course folder every 15 minutes and runs a Python pipeline that calls the Claude API once per source file.

For each source file	myTutor produces
Reference note	Summary, key points, link back to the original file
Core notes	One note per key concept, linked to notes already in the vault
Case notes	One per real organisation discussed
Quiz	5 multiple-choice questions; plus a 20-question practice set per teaching day

2. Architecture



Pipeline stages (run.py)

Stage	What it does	Writes
1 retrieve.py	Finds new files in `<course>/<day>/`, skips files already seen (SHA-256), copies them into a dated job folder	`source/`, `manifest.json`, `index.json`, `runs.csv`
2 extract.py	PDF, PPTX and DOCX to clean text; counts words and pages; guesses the kind (slides, reading, case)	`text/*.md`, `extract.json`

3 generate.py	Calls Claude once per source with the vault's existing note titles; tool use returns notes and quiz as structured data; builds the day's 20-question set	`notes/` , `quiz/*.json` , `generate.json` , `generate_runs.csv`
4 file.py	Copies notes into the Obsidian vault; never overwrites a note you edited (SHA check); picks a random 10-note spot-check sample	vault notes, `filed.json` , `file_runs.csv` , `quality_audit.csv`
5 practice.py	Builds one offline HTML practice page from all saved quiz sets; options are shuffled on screen	`practice.html`

> **Orchestration:** n8n's workflow has two triggers (a 15-minute schedule and a manual *Run now*) feeding one Execute Command node, then a Code node that reads the `SUMMARY {json}` line printed by run.py. Windows Task Scheduler running `run_mytutor.bat` is the fall-back.

3. Prerequisites

You need	Why	Check
Windows 10/11 PC	Commands, .bat files and paths are Windows-style	—
Python 3.11 or later	Runs the pipeline (built and used on 3.12)	`py --version`
Node.js 24 or later	n8n 2.x refuses to run on Node 20	`node -v`
n8n (npm)	Schedules and runs myTutor	`n8n --version`
Anthropic API key	Claude writes the notes; billed per token	Anthropic console
OneDrive app, signed in	Syncs the course folder to disk	Green ✓ on files
Obsidian, with a vault	Where notes are filed and read	Vault opens
Git (optional)	To clone the repository	`git --version`

> **Skills assumed:** using Command Prompt, editing a JSON file, and installing desktop apps. No programming is needed to run myTutor; Python is needed only to change it.

4. Technology stack and versions

Versions in use for the experiment, checked 28 October 2026. "Minimum" is what the code requires.

Component	Version used	Minimum	Role
Windows	11	10	Host OS
Python	3.12.2	3.11	Pipeline runtime
Node.js	24.22.1	24	Runtime for n8n
n8n	2.44.6	2.x	Scheduler and orchestration
Claude model	claude-sonnet-5-5	—	Note and quiz generation (tool use)
anthropic (Python SDK)	1.15.0	1.0	Claude API client

pymupdf	1.28.2	1.24	PDF text extraction
python-pptx	1.0.2	0.6.23	PowerPoint extraction
python-docx	1.2.0	1.1	Word extraction
msal, requests	optional	1.28, 2.31	Only for `"mode": "graph"` (OneDrive download via Microsoft Graph)
Obsidian	1.14.5	any recent	Note vault and graph view
Practice page	HTML + JavaScript, no libraries	any modern browser	Offline quiz

Exact library versions installed on a machine: `py -m pip show anthropic pymupdf python-pptx python-docx`.

Cost reference

Prices are set in `config.json` (`price_per_mtok`: input US$2, output US$10 per million tokens) and logged per source. Measured: about US$0.10 per file; 5 Day 9 files cost US$0.48; Day 1 (10 files, 34,255 words) cost US$1.01.

5. Installation and setup

5.1 Install the prerequisites

```
winget install Python.Python.3.12        (or python.org; tick "Add to PATH")
winget install OpenJS.NodeJS             (Node 24 or later)
npm install -g n8n
```

5.2 Get the code and libraries

```
cd %USERPROFILE%
git clone https://github.com/henrynawaung/mytutor.git
cd mytutor
py -m pip install -r requirements.txt
py -m unittest discover tests            (expect: OK, 67 tests)
```

5.3 Set the API key (environment variable only)

```
setx ANTHROPIC_API_KEY "your-key-here"
```

Open a new Command Prompt so the key is visible. Set a monthly spending limit in the Anthropic console.

5.4 Create config.json

Copy `config.example.json` to `config.json` and set the paths (section 6).

5.5 Prepare OneDrive

Create a folder per course with a sub-folder per teaching day, for example `MyTutor\MBAM601_...\Day 9\`. Right-click the course folder and choose **Always keep on this device**, so files are on disk rather than cloud-only.

5.6 First run by hand

```
py run.py --dry-run                    (which day folders would run)
py run.py --file --sample 10           (process, file into Obsidian, pick a sample)
py practice.py --open                  (open the quiz page)
```

5.7 Automate with n8n

1. Run `start-n8n.bat`. It sets `NODES_EXCLUDE=[]`, because n8n 2.x disables the Execute Command node by default.
2. Open `http://localhost:5678`, then **Import from file** › `n8n_mytutor_workflow.json`.
3. Check the Run myTutor command path, click **Execute workflow** once, then **Publish** so the schedule runs.
4. Fall-back: Task Scheduler running `run_mytutor.bat` every 15 minutes.

> The laptop must be on, n8n running and OneDrive synced for scheduled runs. Runs started by the schedule show without a flask icon in n8n's Executions list.

6. Configuration reference (config.json)

Key	Default	Meaning
`mode`	synced	`synced` reads the OneDrive folder on disk; `graph` downloads via Microsoft Graph (needs an app registration)
`synced_inbox`	—	Folder holding `<course>/<day>/` sub-folders
`jobs_root`	~/mytutor/jobs	Job folders, logs and `practice.html`
`vault_path`	—	Obsidian vault folder
`obsidian_vault_name`	vault folder name	Used in *Open in Obsidian* links
`term`, `term_label`	—	Term tags, e.g. T3 / 26T3; used to find course folders in the vault
`vault_folders`	Ref, Core, Cases, Quizzes, Theme	Sub-folder names inside each course/day
`model`	claude-sonnet-5-5	Claude model
`price_per_mtok`	2.0 / 10.0	Input / output price per million tokens, for cost logging
`max_output_tokens`	12000	Cap per Claude call; a cut-off response is reported as an error
`extensions`	.pdf .pptx .docx .txt .md	File types read; others (images, .zip, .rar) are ignored
`max_file_mb`	100	Larger files are skipped
`settle_minutes`	3	A day folder changed more recently than this waits for the next run
`practice_questions`	20	Size of each day's practice set

7. Running and commands

Command	What it does
`py run.py --file --sample 10`	Full run: all stages, file into vault, 10-note sample (what n8n runs)
`py run.py --dry-run`	List day folders that would be processed
`py generate.py --day "Day 9" --estimate`	Token and cost estimate only
`py generate.py --day "Day 9" --only "Kowhai" --force`	Regenerate one source
`py generate.py --day "Day 10" --rebuild-set`	Rebuild the day's 20-question set (after adding files later)
`py file.py --day "Day 9" --dry-run`	Show what filing would change
`py file.py --day "Day 9" --sample 10 --no-file`	Pick a new spot-check sample only
`py practice.py --open`	Rebuild and open the practice page

8. Data, logs and folders

```
mytutor/
  run.py retrieve.py extract.py generate.py file.py practice.py jobs.py
  config.json            your settings (not in Git)
  jobs/
    index.json           SHA-256 of every file already retrieved
    runs.csv             one row per retrieve
    generate_runs.csv    per source: tokens, cost, seconds, notes, links to existing notes
    file_runs.csv        per filing: filed, updated, unchanged, kept as you edited
    quality_audit.csv    10 sampled notes per run, five structural checks
    run_log.csv          one row per run.py run (incl. scheduled checks)
    practice.html        offline quiz page
    <date>_<course>-day-N/
      source/  text/  notes/  quiz/  manifest.json  extract.json  generate.json
```

Vault layout: `<Course folder>/<Day N>/{Ref, Core, Cases, Quizzes}` plus a journal stub left for the student.

KPIs come from these logs

KPI	Source
Time per source	`generate_runs.csv` seconds + timed review
Note quality	`quality_audit.csv` five checks
Links per note	`generate_runs.csv` links_to_existing ÷ notes
Recall	Practice page score, copied to the log

9. Testing

```
py -m unittest discover tests
```

67 unit tests cover retrieval (duplicates, settling, size limits), extraction, response normalisation, note writing and linking, filing without overwriting edits, practice-set building and run orchestration. The answer-shuffle fix was retested with an automated browser on the real Day 9 set (`docs/evidence/retest/`): option B fell from 55% to 24% of correct answers over 200 runs, with marking unchanged.

10. Security and responsible use

> **API key:** only in the `ANTHROPIC_API_KEY` environment variable. Never in `config.json`, a text file or a commit. `.gitignore` blocks `config.json`, `key.txt`, `.env` and `jobs/`.

- **Course material** is sent to an AI service: process only files you are licensed to use and follow your institution's AI guidance. Course files are never committed to the repository.
- **Execute Command** in n8n runs shell commands; keep n8n on `localhost` and do not expose port 5678.
- **Graph mode** caches a token in `jobs/.token_cache.json`; keep it private.
- **Accuracy:** notes pass structural checks automatically; factual accuracy needs a human spot-check (sample audit: 31 of 32 statements supported).

11. Troubleshooting

Symptom	Fix
`py: can't open file ...\run.py`	Run `cd /d %USERPROFILE%\mytutor` first.
"Could not find platform independent libraries"	Harmless Python warning; ignore.
Day folder not processed	Files are cloud-only: set **Always keep on this device**; or a file changed in the last 3 minutes, so wait for the next run.
Read summary says "Nothing new"	Files already processed (by SHA). Use `--force` on generate.py to redo a source.
n8n: `EBADENGINE` or will not start	Upgrade Node.js to 24 or later.
n8n: Execute Command node missing / "?"	Start with `start-n8n.bat` (sets `NODES_EXCLUDE=[]`).
n8n window blank, page "Cannot GET /"	Still starting; wait for "Editor is now accessible". Press Enter if the window is paused.
Schedule never runs	The workflow is not **Published**.
`ANTHROPIC_API_KEY is not set`	Run `setx` again and open a new window; restart n8n so it sees the key.
"Claude returned no usable notes"	Rerun that source with `--only "name" --force`; blank templates or prompt files make poor sources.
Practice page missing new questions	`py generate.py --day "Day N" --rebuild-set`, then `py practice.py`.

Quiz notes flagged for no source link	Fixed in current version; rerun `file.py`.

12. Known limits and roadmap

Limit	Planned change
No OCR for scanned PDFs	Add OCR before extraction
Long books compress to few concepts	Split readings by section
No case notes for fictional teaching cases	Teaching-case rule in the prompt
Quiz distractors sometimes weak (AI review 72/100)	Prompt now asks for plausible distractors and a 20/50/30 recall/application/analytical mix; next: a validation step against the source before publishing
Scores kept in the browser only	Save results by concept to a file for learning tracking
Single user, single laptop	Shared vault and access control for team or company use

13. Key prompts and the manual test template

Full text of both prompts is in `docs/prompts/PROMPTS.md`.

Prompt	Used for	Where
Manual baseline prompt	The "before" process: pasted into an AI chat, one new chat per file, file attached	`docs/prompts/PROMPTS.md` §1; built by the workbook's Prompt sheet
myTutor system prompt	Every automated call to the Claude API, with the `save_notes` tool	`generate.py` (`SYSTEM_PROMPT`); `docs/prompts/PROMPTS.md` §2

Manual test workbook (docs/manual-test/)

File	Use
`myTutor_Manual_Test_Template.xlsx`	Blank, to run your own manual vs myTutor comparison. Sheets: Start, Steps (each step marked inside or outside the stopwatch), Prompt (fill 3 yellow cells, copy C11:C50), Log, Column guide, Review tally (5 checks per note), Summary (KPIs and % change, calculated)
`myTutor_Manual_Test_Day9_results.xlsx`	The completed Day 9 test behind the deck's results

Manual baseline prompt (template)

Replace `{COURSE}`, `{DAY}`, `{DATE}`, `{FILE}`, `{KIND}` (slides / reading / case) and `{CONCEPTS}` (3 to 6, comma separated).

```
I am attaching one source for my MBA course. Turn it into Obsidian notes in Markdown.

Course: {COURSE}   Day: {DAY}   Date: {DATE}
Source file: {FILE}
Concepts I want covered: {CONCEPTS}

Write these notes. Put each note in its own code block, with its file name on the line above
the block.

1. ONE reference note, named after the source.
   Frontmatter:
   ---
   type: reference
   course: {COURSE}
   day: {DAY}
   kind: {KIND}
   source: "{FILE}"
   created: {DATE}
   tags: [reference]
   ---
   Sections: Overview (3 sentences), Key points (5 to 8, in the source's order), Frameworks and
models (each linked to its core note), Questions I should be able to answer (3), Related (links
to every core, case and quiz note below).

2. THREE TO SIX core notes, one per concept from my list, named after the concept in title
case.
   Frontmatter: type: core, course: {COURSE}, first_seen: "{FILE}", created: {DATE}, tags:
[core]
   Body: Definition, Why it matters, Example from the source, Related concepts, Cases that
illustrate it, Source: [[<reference note title>]]. Under 200 words each.

3. ONE case note per real organisation discussed as a case, ONLY if the source has one. Name:
"Case - <Organisation>".
   Frontmatter: type: case, course: {COURSE}, day: {DAY}, organisation, source: "{FILE}",
created: {DATE}, tags: [case]
   Body: Context, Problem, What they did, Outcome, Concepts illustrated, Source.

4. ONE quiz note, named "Quiz - {COURSE} {DAY} - <source title>".
   5 multiple-choice questions testing understanding, not wording. 4 options each, one correct.
Give the answer and a 1-2 sentence explanation for each, and link the core note it comes from.

Rules:
- Link notes to each other with [[Exact Title]].
- Use only the source. No outside facts. If something is unclear, write "Unclear in source".
- Keep the source's own names for models and frameworks.
```